

# Отчёт о выполнении лабораторной работы №1 «Поиск частых наборов объектов (алгоритм Apriori)»

Дисциплина: Основы интеллектуального анализа данных

Выполнил: Andrey Zamyatin, группа ET-442, ЮУрГУ

## 1. Формулировка задания

Разработать программу, выполняющую поиск частых наборов объектов в заданном наборе данных с помощью алгоритма Apriori (или одной из его модификаций). Список результирующих наборов должен содержать сами наборы и значение поддержки для каждого набора. Параметрами программы являются набор данных, порог поддержки и способ упорядочивания результирующего списка наборов (по убыванию значения поддержки или лексикографическое).

Требуется провести эксперименты на наборе данных `baskets.csv` (сведения о покупках в супермаркете), варьируя пороговое значение поддержки (1%, 3%, 5%, 10%, 15%), и визуализировать результаты в виде двух диаграмм: сравнение быстродействия алгоритма на фиксированном наборе данных при изменяемом пороге поддержки; количество частых наборов объектов различной длины на фиксированном наборе данных при изменяемом пороге поддержки.

## 2. Репозиторий с исходными материалами

Исходный код, набор данных и остальные материалы лабораторной работы лежат в каталоге `lab1` репозитория на GitHub: [https://github.com/Trenj/data\\_mining/tree/main/lab1](https://github.com/Trenj/data_mining/tree/main/lab1). Состав каталога `lab1`:

- `apriori_market_basket.ipynb` реализация алгоритма Apriori, проведение экспериментов и построение графиков
- `baskets.csv` исходный набор данных о покупках (7501 транзакция, 115 уникальных товаров, кодировка `cp1251`)
- `runtime_vs_support.png` график зависимости времени выполнения от порога поддержки
- `itemset_counts_by_length.png` график количества частых наборов разной длины от порога поддержки
- `Task.txt` текст задания

### 2.1. Краткое описание реализации

Алгоритм Apriori реализован с нуля на Python, без сторонних библиотек для самого поиска частых наборов. Функция `apriori(transactions, min_support, sort_by)` принимает список транзакций, порог поддержки (доля от 0 до 1) и способ сортировки результата (параметр `sort_by` принимает значение `support_desc` для сортировки по убыванию поддержки или `lexicographic` для лексикографической сортировки), а возвращает список пар (набор объектов, значение поддержки). Код разбит на несколько функций: `get_frequent_items` находит частые наборы, состоящие из одного товара; `generate_candidates` строит кандидатов следующего размера, отбрасывая тех, у кого есть нечастое подмножество (на этом свойстве антимонотонности поддержки построен Apriori); `count_support` считает поддержку кандидатов по всем транзакциям. Основной цикл повторяет генерацию кандидатов и подсчёт поддержки, пока находятся новые частые наборы.

### 2.2. Проверка корректности (порог 10%)

На пороге поддержки 10% алгоритм находит 7 частых наборов, все длины 1. Ниже показан результат в обоих режимах сортировки: набор данных один и тот же, различается только порядок.

#### По убыванию поддержки

| Товар            | Поддержка |
|------------------|-----------|
| минеральная вода | 0.2384    |
| макароны         | 0.1880    |
| яйца             | 0.1797    |
| картофель-фри    | 0.1709    |
| шоколад          | 0.1638    |
| зеленый чай      | 0.1321    |
| молоко           | 0.1296    |

#### Лексикографически

| Товар            | Поддержка |
|------------------|-----------|
| зеленый чай      | 0.1321    |
| картофель-фри    | 0.1709    |
| макароны         | 0.1880    |
| минеральная вода | 0.2384    |
| молоко           | 0.1296    |
| шоколад          | 0.1638    |
| яйца             | 0.1797    |

### 3. Результаты экспериментов

Эксперименты проведены на наборе данных baskets.csv (7501 транзакция, 115 уникальных товаров, средняя длина транзакции 3.91 товара) при пороге поддержки 1%, 3%, 5%, 10% и 15%. Для каждого порога измерено время работы алгоритма и число найденных частых наборов в разбивке по длине. Сводные результаты собраны в таблице ниже.

| Порог, % | Всего наборов | Время, с | длина 1 | длина 2 | длина 3 | длина 4 | длина 5 |
|----------|---------------|----------|---------|---------|---------|---------|---------|
| 1        | 261           | 9.944    | 74      | 170     | 17      | 0       | 0       |
| 3        | 55            | 1.099    | 36      | 19      | 0       | 0       | 0       |
| 5        | 28            | 0.460    | 25      | 3       | 0       | 0       | 0       |
| 10       | 7             | 0.048    | 7       | 0       | 0       | 0       | 0       |
| 15       | 5             | 0.030    | 5       | 0       | 0       | 0       | 0       |

#### 3.1. Быстродействие алгоритма

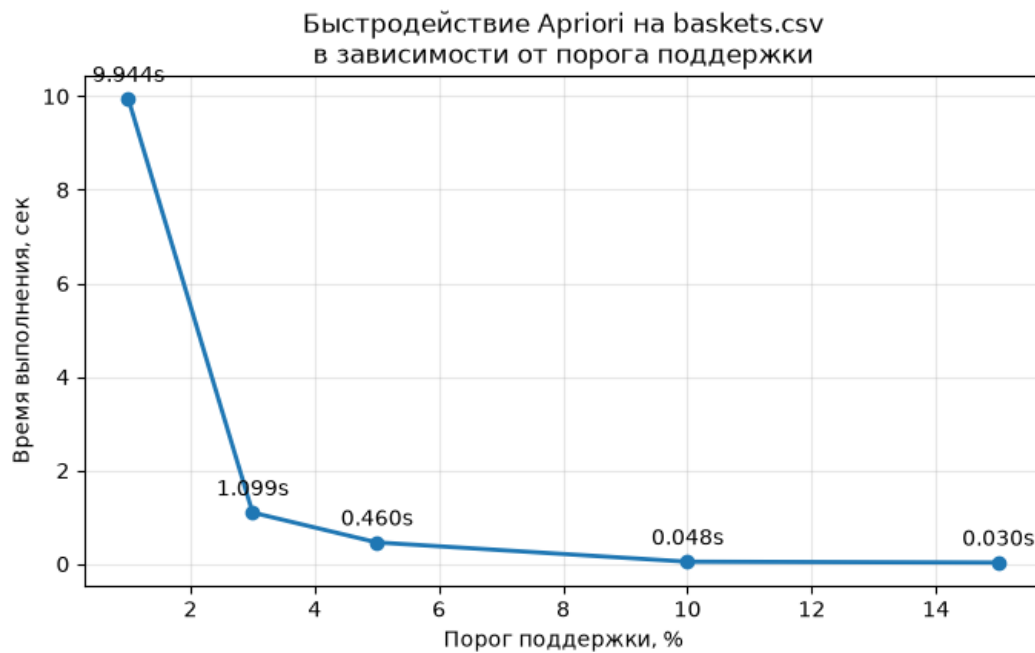


Рисунок 1. Время выполнения Apriori на baskets.csv от порога поддержки.

### 3.2. Количество частых наборов по длине

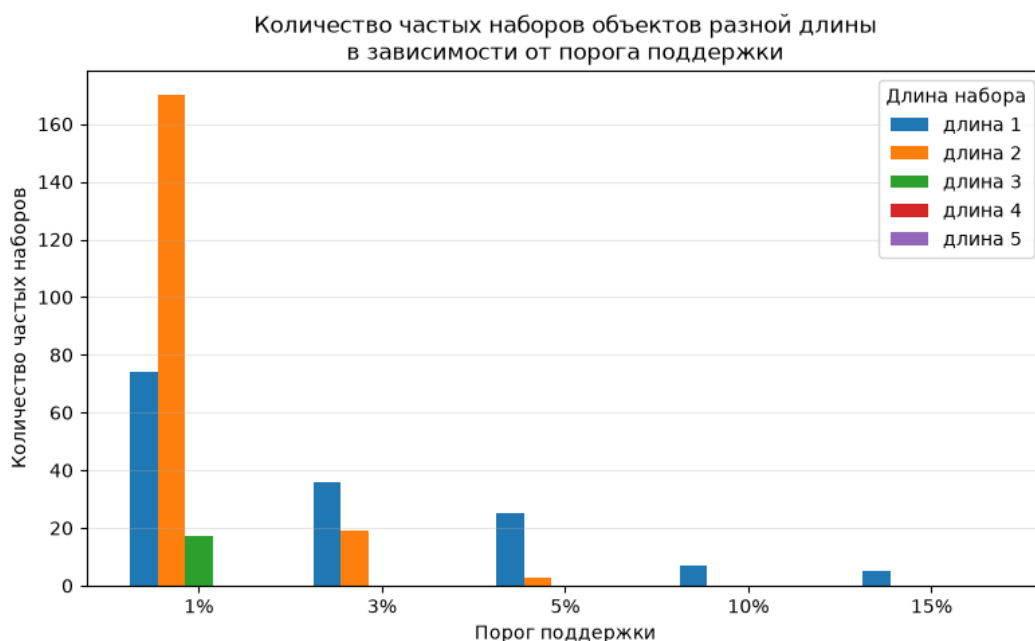


Рисунок 2. Количество частых наборов объектов разной длины от порога поддержки.

## 4. Анализ и выводы

С понижением порога поддержки количество найденных частых наборов резко растёт: с 5 наборов при пороге 15% до 261 набора при пороге 1%. Основной вклад в этот рост дают наборы длины 2 и 3. При порогах 10% и 15% в результат попадают только отдельные товары, а при более низких порогах (от 1% до 5%) уже видны совместные покупки пар товаров, а при пороге 1% видны и тройки.

Время выполнения растёт вместе с числом наборов: с 0.03 с при пороге 15% до почти 10 с при пороге 1%. Причина в том, что при низком пороге частыми на первом шаге оказываются больше отдельных товаров, из-за чего Apriori строит и проверяет намного больше кандидатов на шагах 2 и 3, пересчитывая для каждого поддержку по всем 7501 транзакциям.

Из результатов виден обычный для частотного анализа компромисс: более низкий порог поддержки даёт более подробную картину совместных покупок (минеральная вода, макароны и яйца остаются среди самых популярных как по отдельности, так и в парах при низких порогах), но требует заметно больше вычислений из-за роста числа кандидатов. Порог имеет смысл выбирать под задачу: если нужен детальный разбор совместных покупок, порог берут ниже, а если достаточно списка самых ходовых товаров, то выше.